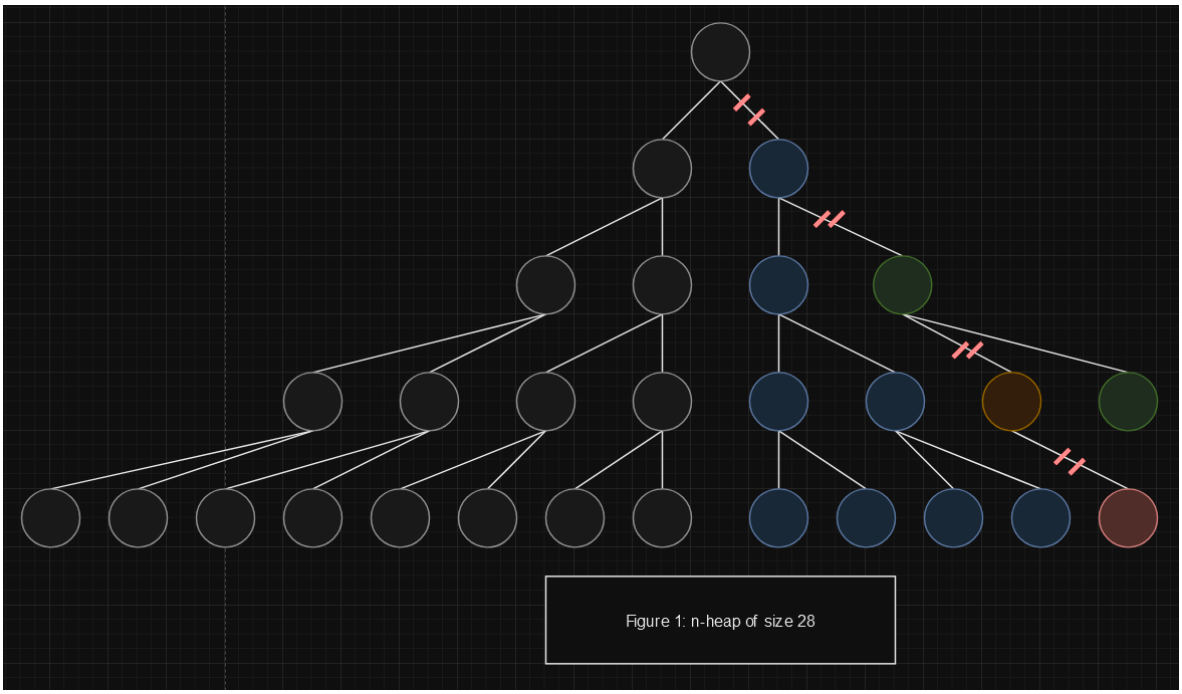
Leo Keenan - 101222357

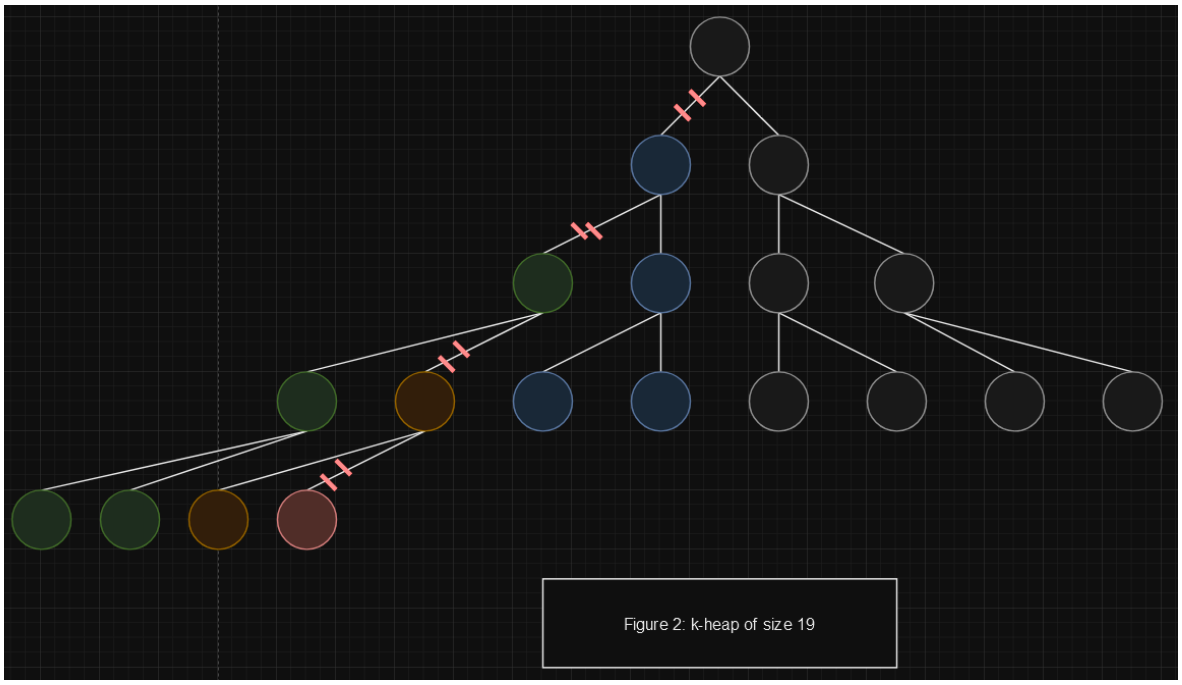
Assignment 2 - Leo Keenan

Question 1



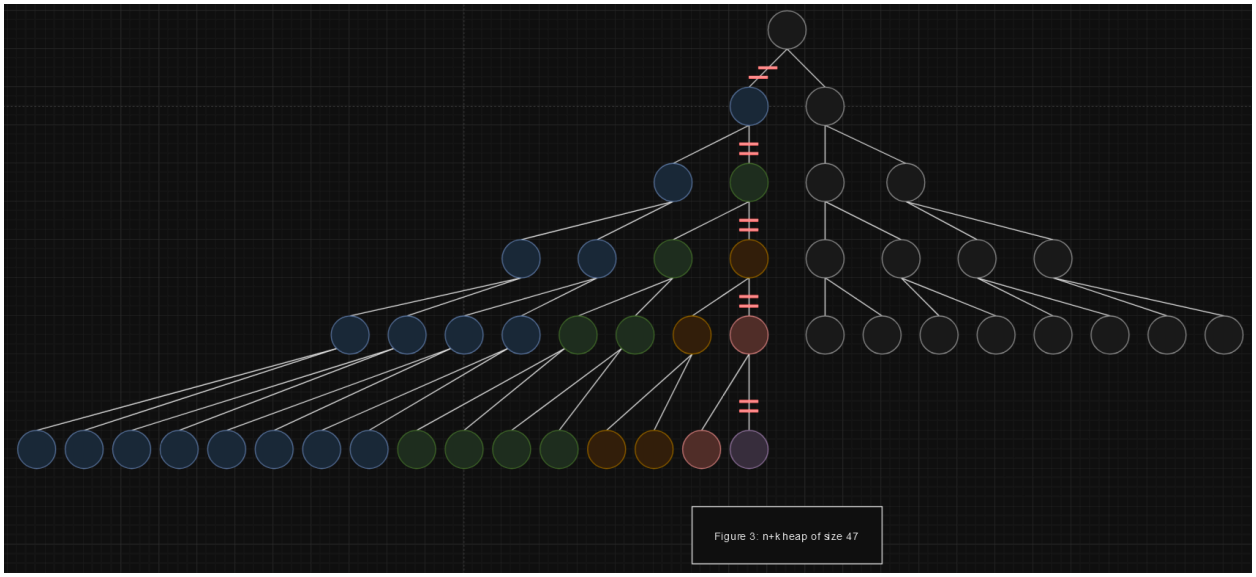
2^4 = 1, 2^3 = 1, 2^2 = 0, 2^1 = 1, 2^0 = 2

Descriptor for n-heap of size 28 is (1,1,0,1,2)



2^3 = 1, 2^2 = 2, 2^1 = 1, 2^0 = 1

Descriptor for k-heap of size 19 is (1,2,1,1)



2^4 = 2, 2^3 = 1, 2^2 = 1, 2^1 = 1, 2^0 = 1

Descriptor for $k + n$ heap of size 47 is (2,1,1,1,1)

To form the require pennants of $(n + k)$−heap using the pennants of n-heap and the k -heap, we do the following:

- **One** pennant of size **1** is taken from **k-heap**
- **Two** pennants of size **1** are taken from the **n-heap** and merged into a pennant of size **2**
- **Two** pennants of size **2** are taken from the **k-heap and n-heap** and merged into a pennant of size **4**
- **Two** pennants of size **4** are taken from the **k-heap** and merged into a pennant of size **8**
- **Two** pennants of size **8** are taken from the **k-heap and n-heap** and merged into a pennant of size **16**

Leo Keenan - 101222357

Question 2

Creation:

- Use the deterministic Select algorithm to find the $\lfloor \frac{n}{2} \rfloor$ order statistic and then partition the remaining elements around it into sets S_L and S_R. This takes $O(n)$ time.
- Within S_R, use the Select algorithm to find the $\lfloor \frac{3n}{4} \rfloor$ order statistic and then partition the remaining elements of S_R around it into sets S_{RL} and S_{RR}. This takes $O(n)$ time.
- Create a pointer-based data structure in which the $\lfloor \frac{n}{2} \rfloor$ order statistic is the root and has two children. Set the left child to the root of a min-max heap on the set S_L. Set the right child to the $\lfloor \frac{3n}{4} \rfloor$ order statistic, which itself will have two children. Set its left child to a min-max heap on the set S_{RL} and its right child to a min-max heap on the set S_{RR}. Each Heap can be created in $O(n)$ time.
- The total construction time therefore takes $O(n)$ time.

Access:

- To access the 1 order statistic start from the root, go to the left child.
- To access the $\lfloor \frac{3n}{4} \rfloor$ order statistic, go to the right child of the root.
- To access the $\lfloor \frac{n}{2} \rfloor$ order statistic, take the root.
- To access the n order statistic, go to the root's right child's right child and query the maximum of the heap. All queries used on heaps here cost $O(1)$ and the operations to walk over pointers to the appropriate children also take $O(1)$ time. Therefore, each of these access operations take $O(1)$ time.
- To list all elements between the $\lfloor \frac{n}{2} \rfloor$ and $\lfloor \frac{3n}{4} \rfloor$ order statistics, start from the root, go to the right child, then go to the left child, then traverse the heap rooted at this location. Since this heap contains the elements of S_{RL}, this will give exactly the elements between $\lfloor \frac{n}{2} \rfloor$ and $\lfloor \frac{3n}{4} \rfloor$ order statistics. Heap traversal takes $O(n)$ time and both the traversal and the prior pointer accesses require no element comparisons.

Insertion:

- To insert an element, we identify which of the sets S_{RL}, S_{RR}, or S_L it belongs in using comparisons to the $\lfloor \frac{n}{2} \rfloor$ and $\lfloor \frac{3n}{4} \rfloor$ order statistics and when we insert it into the appropriate heap. Depending on the change in heap sizes, the $\lfloor \frac{n}{2} \rfloor$ and $\lfloor \frac{3n}{4} \rfloor$ order statistics may need to be updated. Due to the use of mini-max heaps, the elements which come immediately before and after those order statistics can be accessed in constant time.
- To rebalance, we delete the new order statistic from its original heap, replace the old order statistic with it and then insert the old order statistic into the heap of the appropriate set to keep the structure balanced.
- Each operation for a deletion or insertion takes $O(logn)$ time and there are a constant number of these operations required.

Deletion:

- Deletion of the order statistic can be done by deleting the minimum value from the min-max heap on S_{RL}.
- Deleting the n order statistic can be done by deleting the maximum value from the min-max heap on S_L. These operations may require rebalancing which can be done as described for insertion.
- Deleting the $\lfloor \frac{3n}{4} \rfloor$ and $\lfloor \frac{n}{2} \rfloor$ order statistics can be done by determining which element from the sets S_{RL}, S_{RR}, or S_L would be required to replace it upon deletion, then deleting the appropriate element from its heap and placing that value at the order statistic positions. Each of these operations require $O(logn)$ time

Leo Keenan - 101222357

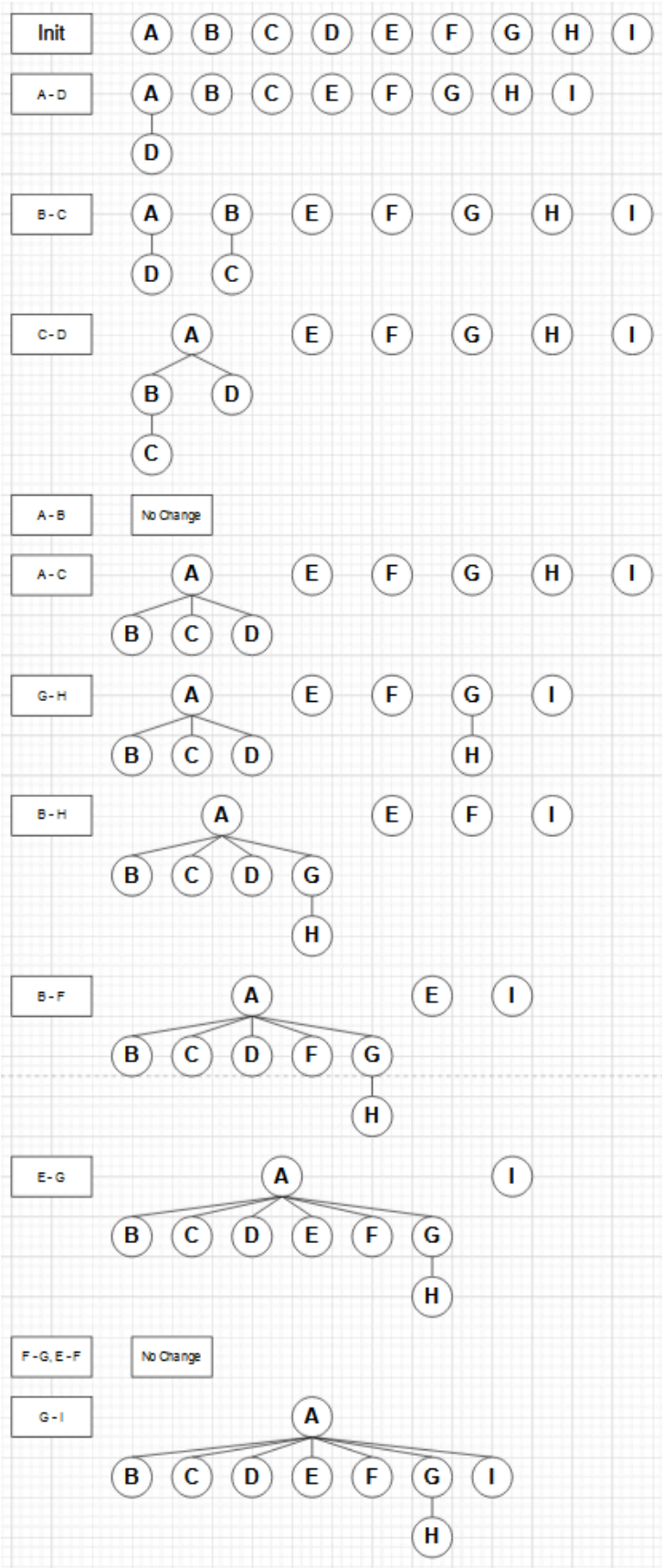
Question 3

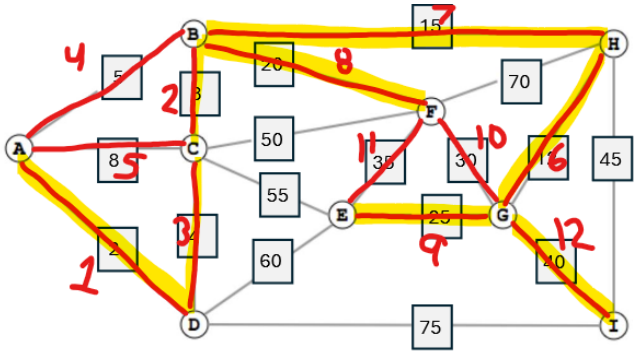
Execution of Prim's Algorithm

Step #	**set S**	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
Init	$\{\}$	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
1	$\{A\}$		$5/A$	$8/A$	$2/A$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
2	$\{A, D\}$		$5/A$	$4/D$		$60/D$	∞/nil	∞/nil	∞/nil	$75/D$
3	$\{A, C, D\}$		$3/C$			$55/C$	$50/C$	∞/nil	∞/nil	$75/D$
4	$\{A, B, C, D\}$					$55/C$	$20/B$	∞/nil	$15/B$	$75/D$
5	$\{A, B, C, D, H\}$					$55/C$	$20/B$	$12/H$		$45/H$
6	$\{A, B, C, D, G, H\}$					$25/G$	$20/B$			$40/G$
7	$\{A, B, C, D, F, G, H\}$					$25/G$				$40/G$
8	$\{A, B, C, D, E, F, G, H\}$									$40/G$
9	$\{A, B, C, D, E, F, G, H, I\}$									

Leo Keenan - 101222357

Krustkal's Algorithm





Leo Keenan - 101222357

Question 4

a)
Producing MST
Because the same constant is added to each of the edge-weights in G' the relative order of the edges by their weights does not change. Let's compare two edges e_1 amd e_2
$w'(e_1) = w(e_1) + c$ and $w'(e_2) = w(e_2) + c$
$w'(e_1) < w'(e_2) \Leftrightarrow w(e_1) < w(e_2)$
therefore, krustkal's algorithm selects the same edges for G' as it would for G, and the order remains unaffected by adding a constant c to each edge weight.

Weights of MST for G' compared to G:
Let T be the MST of G, and the total weight of the MST is the sum of edge weights:
$w(T) = \Sigma_{e \epsilon T} w(e)$
The total weight of the MST for G', denoted $w'(T)$, is:

$$w'(T) = \sum_{e \epsilon T} w'(e) = \sum_{e \epsilon T} (w(e) + c) = w(T) + c \cdot |T|$$

Since $|T| = |V| - 1$ (the number of edges in MST is always $|V| - 1$), the weight of the MST for G' is $w'(T) = w(T) + c \cdot (|V| - 1)$.

b)

Let e_{max} be the edge with the maximum weight in the graph G, and suppose e_{max} is part of the MST T.

Proof by contradiction:
Assume removing e_{max} from the MST does not disconnect the graph. This implies there is an alternate path between vertices connect by e_{max} in T. Let this alternate path consist of edges from T, whose weights are all strictly less than $w(e_{max})$, since all edge weights are distinct.

By adding one of the edges from this alternate path to T (after removing e_{max}), we could form a new spanning tree with a smaller total weight, contradicting the fact that T is a minimum spanning tree.

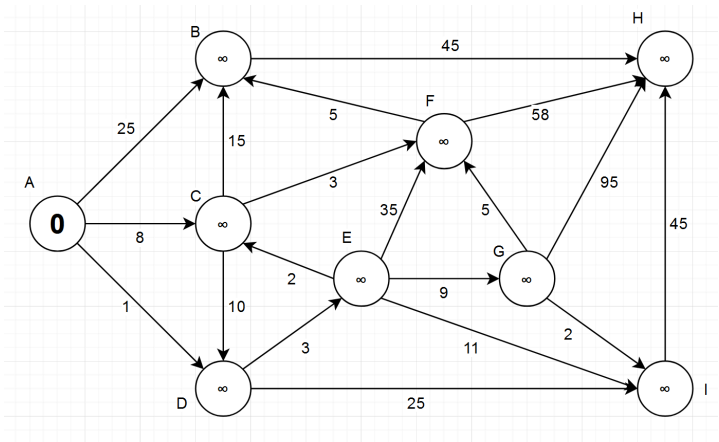
Thus removing e_{max} must disconnect the graph, as there is no alternate path with a smaller weight. Hence, e_{max} is a critical edge for maintaining the connectivity of the MST.

Leo Keenan - 101222357

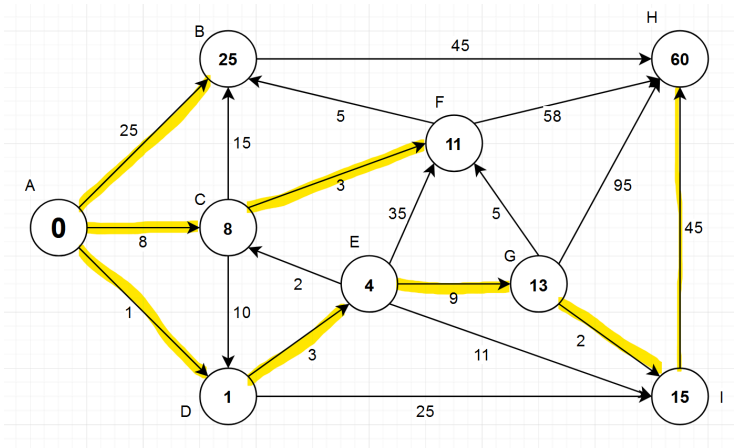
Question 5

Bellman-Ford

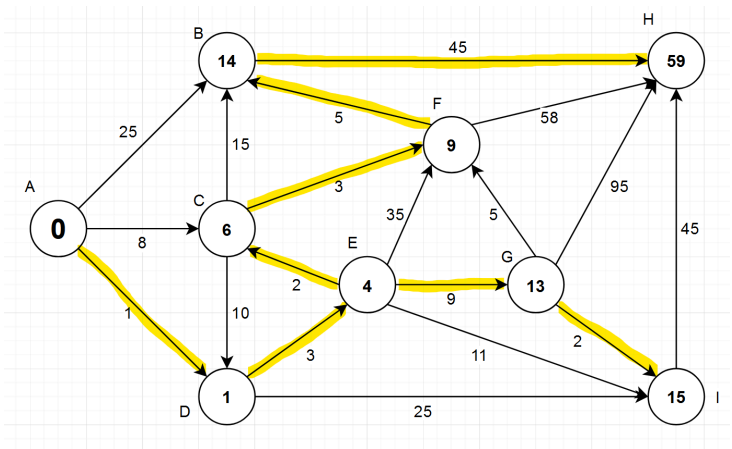
Initial Graph



Iteration 1:



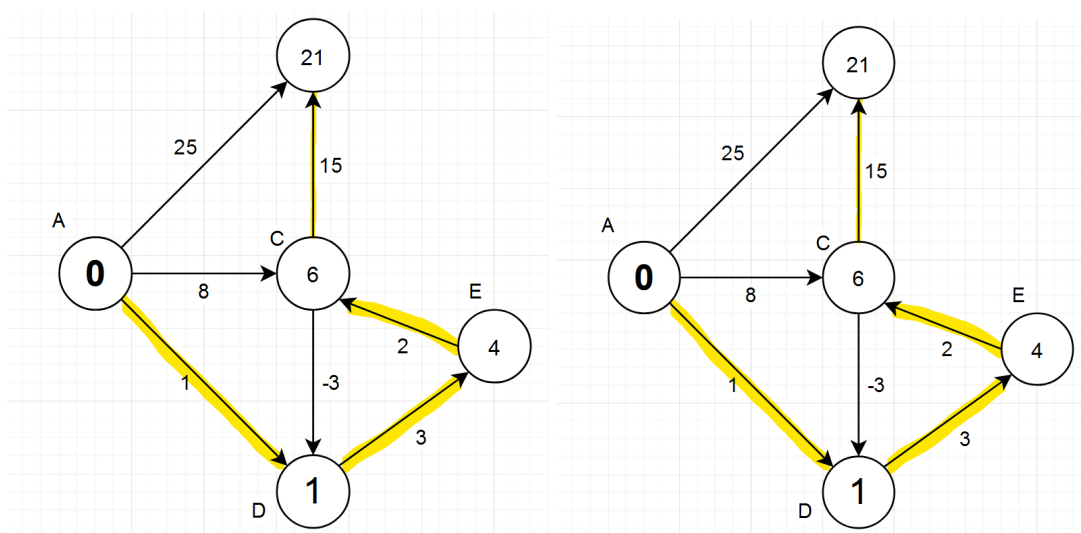
Iteration 2:



Leo Keenan - 101222357

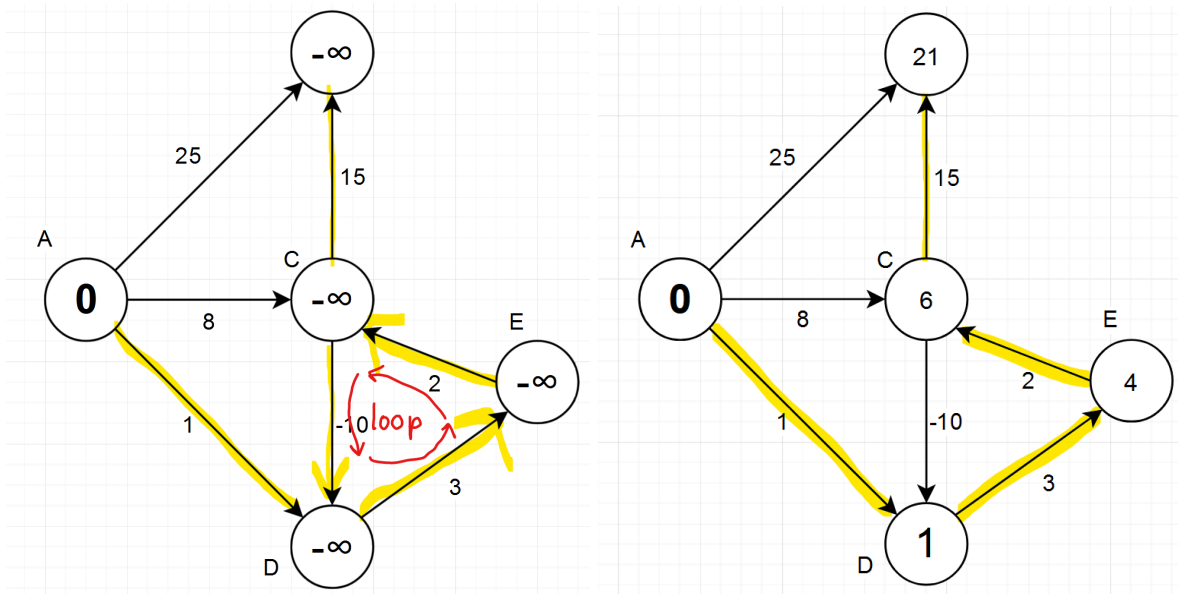
Execution of Dijkstra's Algorithm

Step #	**set S**	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
Init	$\{\}$	0	∞	∞	∞	∞	∞	∞	∞	∞
1	$\{A\}$		$25/A$	$8/A$	$1/A$	∞	∞	∞	∞	∞
2	$\{A, D\}$		$25/A$	$8/A$		$4/D$	∞	∞	∞	$26/D$
3	$\{A, D, E\}$		$25/A$	$6/E$			$39/E$	$13/E$	∞	$15/E$
4	$\{A, C, D, E\}$		$21/C$				$9/C$	$13/E$	∞	$15/E$
5	$\{A, C, D, E, F\}$		$14/F$					$13/E$	$67/F$	$15/E$
6	$\{A, C, D, E, F, G\}$		$14/F$						$67/F$	$15/E$
7	$\{A, B, C, D, E, F, G\}$								$59/B$	$15/E$
8	$\{A, B, C, D, E, F, G, H\}$									$15/E$
9	$\{A, B, C, D, E, F, G, H, I\}$									



Comparison between Bellman-Ford (left) and Dijkstra's Algorithms (right) with edge CD = -3

Leo Keenan - 101222357



Comparison between Bellman-Ford (left) and Dijkstra's Algorithms (right) with edge CD = -10

When changing the edge CD to -3 both algorithms produce the same result, however when it becomes -10 Bellman-Ford enters a loop that keeps repeating every iteration because it going from A->D has a weight of 1, but if you keep looping through E followed by C then back to D the weight will go down by 5 every loop. Dijkstra's on the other hand will not change the weights of a node once it has determined the most efficient path, so even though it sees the loop it has already permanently labeled D with a weight of 1.